Deep Graph Learning
Lời thuyết trình cho 4 người

26 slide · 37 phút trình bày + 3 phút hỏi đáp

Bản này đi cùng PowerPoint và chứa cùng lời nói trong phần Notes. Tên thành viên chưa được cung cấp, nên các phần dùng tên Thành viên 1–4. Nhóm thay bằng tên thật khi phân công. Thời lượng là mốc tập luyện dự kiến, đã dành thời gian cho thao tác demo và tương tác với lớp.

Người	Slide	Thời lượng	Mốc thời gian
Thành viên 1	1–6	8 phút	00:00–08:00
Thành viên 2	7–13	10 phút	08:00–18:00
Thành viên 3	14–20	10 phút	18:00–28:00
Thành viên 4	21–26	9 phút	28:00–37:00
Cả nhóm	Hỏi đáp	3 phút	37:00–40:00

Cách dùng khi tập nói

Đọc phần lời nói để nắm mạch, sau đó tập giải thích bằng cách diễn đạt của mình. Các đoạn “Thao tác và nhịp trình bày” và “Chuyển tiếp” dành cho người trình bày, không đọc nguyên văn cho lớp. Người tiếp theo nên có mặt ở vị trí sẵn sàng trước mốc chuyển phần.

Chuẩn bị demo

Mở sẵn ứng dụng và thử hai mã 444191, 158614. Link triển khai đang private, cần tài khoản chủ sở hữu. Chạy sẵn bản local bằng python -m http.server 8765 --directory dist trong thư mục demo và mở http://localhost:8765. Các số liệu thực nghiệm đã nằm trong slide, không cần huấn luyện lại trên lớp.

Mã nguồn, dữ liệu, checkpoint và hướng dẫn: D:\School\PT HTTM\DeepGraph\demo

01. Deep Graph Learning

Thành viên 1 | 60 giây | Mốc 00:00–01:00

Kính chào thầy và các bạn. Nhóm em trình bày chuyên đề Deep Graph Learning, tập trung vào mạng nơ-ron đồ thị và bài toán phân loại chủ đề bài báo. Trước hết, mời mọi người hình dung hai bài báo có một số từ khóa khá giống nhau. Một bài thường được trích dẫn cùng các công trình về mạng nơ-ron, bài còn lại liên hệ nhiều với lý thuyết học máy. Nếu chỉ nhìn từ khóa, chúng ta có thể khó phân biệt. Vậy các mối liên hệ có giúp dự đoán tốt hơn không?

Đó là câu hỏi xuyên suốt bài trình bày. Nhóm sẽ đi từ cách biểu diễn đồ thị, giải thích phép tính của GCN, so sánh thêm GraphSAGE và GAT, sau đó trình bày thực nghiệm trên Cora và chạy demo. Trong khoảng 37 phút, bốn thành viên lần lượt phụ trách bối cảnh, cơ chế học, thực nghiệm và ứng dụng. Ba phút cuối dành cho trao đổi. Các kết quả trình bày là số liệu từ những lần huấn luyện đã lưu trong mã nguồn, để chúng ta có thể đối chiếu hoặc chạy lại.

Thao tác và nhịp trình bày

Dừng khoảng 5 giây sau câu hỏi mở đầu. Không cần mở demo ngay.

Chuyển tiếp

Trước khi đi vào mô hình, em nối đề tài này với năm chương đã học.

Nguồn đối chiếu

Tài liệu môn Phát triển hệ thống thông minh, Trần Đình Quế, D:/School/PT HTTM/. Chương 2 đối chiếu Assignment2.pdf vì thư mục không có Slide2.pdf. Hình bìa: minh họa khái niệm do Imagegen tạo, không phải đồ thị dữ liệu Cora.

02. Liên hệ với 5 chương đã học

Thành viên 1 | 75 giây | Mốc 01:00–02:15

Deep Graph không bắt đầu bằng việc bỏ những kiến thức cũ. Ở chương một, chúng ta đã học rằng hệ thống thông minh cần biểu diễn dữ liệu phù hợp, phương pháp học và đánh giá thực nghiệm. Đồ thị chính là một lựa chọn biểu diễn khi mối quan hệ giữa các đối tượng có ý nghĩa.

Trong tài liệu bài tập chương hai, quy trình xử lý dữ liệu, chia train, validation, test rồi triển khai đã rất rõ. Sang đồ thị, quy trình đó vẫn giữ, nhưng phải nói thêm mô hình nhìn thấy những cạnh và những nhãn nào. Chương ba cho chúng ta MLP, ReLU, backpropagation và optimizer. GCN vẫn dùng các thành phần đó, chỉ bổ sung bước trộn thông tin hàng xóm trước phép biến đổi đặc trưng.

Chương bốn về CNN gợi ra hai ý quan trọng: khai thác lân cận và chia sẻ trọng số. Với đồ thị, lân cận không còn là ô vuông cố định. Chương năm về Knowledge Graph và truy xuất giúp chúng ta hiểu quan hệ có thể tham gia cả biểu diễn tri thức lẫn xây dựng ứng dụng. Vì vậy, đề tài này nối được phần toán, phần học và phần hệ thống của môn học.

Thao tác và nhịp trình bày

Chỉ lần lượt từng dòng. Nhấn mạnh chương 2 lấy từ Assignment2, không khẳng định đã đọc Slide2.

Chuyển tiếp

Động lực cụ thể là: nhiều dữ liệu thực tế chứa thông tin mà từng dòng độc lập không thể hiện hết.

Nguồn đối chiếu

Tài liệu môn Phát triển hệ thống thông minh, Trần Đình Quế, D:/School/PT HTTM/. Chương 2 đối chiếu Assignment2.pdf vì thư mục không có Slide2.pdf. Slide1.pdf tr.23,25,34,36,60; Assignment2.pdf tr.2,7–10; Slide3.pdf tr.4–24,30; Slide4.pdf tr.3–8,28,33; Slide5.pdf tr.10–14,36,51.

03. Động lực: dữ liệu có quan hệ

Thành viên 1 | 85 giây | Mốc 02:15–03:40

Một dòng dữ liệu thường mô tả một đối tượng. Nhưng trong nhiều bài toán, đối tượng còn nằm trong một mạng quan hệ. Với bài báo, chúng ta có nội dung và mạng trích dẫn. Với hệ gợi ý, chúng ta có đặc trưng người dùng, sản phẩm và những tương tác giữa chúng. Với phân tử, loại nguyên tử chưa đủ, cách các nguyên tử liên kết cũng ảnh hưởng tới tính chất của phân tử.

Nếu mô hình chỉ nhìn từng vector độc lập, phần cấu trúc ấy không trực tiếp đi vào dự đoán. Ta có thể tự tạo các đặc trưng như số hàng xóm hoặc số tương tác, nhưng cách làm đó đòi hỏi thiết kế thủ công. Mạng nơ-ron đồ thị cho phép học biểu diễn từ cả đặc trưng và cấu trúc lân cận.

Tuy nhiên, thêm cạnh không mặc nhiên làm mô hình tốt hơn. Một quan hệ có thể thiếu, sai, hoặc nối hai đối tượng khác nhãn. Nếu giả định của mô hình không phù hợp với kiểu quan hệ, trộn hàng xóm có thể làm mất tín hiệu tốt. Do đó, câu hỏi nghiên cứu của nhóm không phải cứ có graph là dùng GNN, mà là trong dữ liệu và quy trình đã chọn, graph mang lại bao nhiêu thông tin hữu ích so với baseline chỉ dùng đặc trưng.

Thao tác và nhịp trình bày

Mời lớp nêu một ví dụ dữ liệu có quan hệ trong khoảng 10 giây. Dùng mạng trích dẫn để dẫn lại bài toán chính.

Chuyển tiếp

Muốn đưa các quan hệ này vào mô hình, ta cần biểu diễn chúng bằng những đại lượng cụ thể.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216

04. Biểu diễn đồ thị và thuật ngữ

Thành viên 1 | 90 giây | Mốc 03:40–05:10

Ta ký hiệu một đồ thị là G gồm tập nút V và tập cạnh E. Ma trận X chứa đặc trưng, mỗi hàng ứng với một nút. Ma trận kề A mô tả nút nào có liên kết với nút nào. Nhãn y là giá trị cần dự đoán và chỉ một phần nhãn được phép dùng trong quá trình học.

Đồ thị có thể có hướng hoặc vô hướng. Nó cũng có thể đồng nhất, nghĩa là các nút và cạnh cùng loại, hoặc không đồng nhất với nhiều kiểu thực thể và quan hệ. Trong thí nghiệm Cora của nhóm, mọi nút đều là bài báo. Quan hệ trích dẫn vốn có hướng, nhưng nhóm đối xứng hóa cạnh để tạo đồ thị vô hướng cho các mô hình đang xét. Đây là quyết định tiền xử lý cần ghi rõ.

Có ba khái niệm dễ bị dùng lẫn. Knowledge Graph là cách tổ chức tri thức bằng thực thể và quan hệ. GNN là họ mô hình học biểu diễn trên đồ thị. GraphRAG là cách sử dụng cấu trúc đồ thị trong truy xuất để hỗ trợ sinh câu trả lời. Một hệ thống có thể kết hợp cả ba, nhưng dùng Knowledge Graph hay GraphRAG không có nghĩa là đã huấn luyện GNN. Trong buổi này, phần học sâu nằm ở GNN và các ma trận trọng số của nó.

Thao tác và nhịp trình bày

Chỉ bảng ký hiệu trước, rồi mới phân biệt ba thuật ngữ. Không sa đà vào các loại graph ngoài phạm vi.

Chuyển tiếp

Với cùng một cấu trúc graph, đầu ra cần dự đoán có thể nằm ở những cấp độ khác nhau.

Nguồn đối chiếu

Tài liệu môn Phát triển hệ thống thông minh, Trần Đình Quế, D:/School/PT HTTM/. Chương 2 đối chiếu Assignment2.pdf vì thư mục không có Slide2.pdf. Chương5/Slide5.pdf tr.10–14,36.
Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907

05. Ba cấp độ bài toán trên graph

Thành viên 1 | 80 giây | Mốc 05:10–06:30

Ở cấp độ nút, mô hình dự đoán một nhãn cho từng nút. Ví dụ của nhóm là dự đoán chủ đề của từng bài báo trong Cora. Đầu ra tương ứng có số hàng bằng số nút, và mỗi hàng có bảy điểm số cho bảy chủ đề.

Ở cấp độ cạnh, mô hình dự đoán khả năng có liên kết hoặc loại liên kết giữa hai nút. Một hệ gợi ý có thể hỏi người dùng này có khả năng tương tác với sản phẩm kia hay không. Việc đánh giá cần lưu ý cạnh nào được giữ lại để kiểm thử và cách lấy mẫu cạnh âm.

Ở cấp độ đồ thị, cả một graph được xem như một mẫu. Chẳng hạn một phân tử là một đồ thị, và đầu ra có thể là một thuộc tính của phân tử đó. Lúc này thường cần một bước readout hoặc pooling để gom biểu diễn nhiều nút thành một vector đại diện cho cả graph.

Trong 40 phút, nhóm chỉ thực nghiệm sâu bài toán phân loại nút. Hai loại còn lại giúp xác định phạm vi và cho thấy hướng mở rộng. Chúng cần thiết kế nhãn, chia dữ liệu và thước đo khác, nên không thể lấy nguyên accuracy của Cora để kết luận cho mọi bài toán đồ thị.

Thao tác và nhịp trình bày

Nhấn vào hàng phân loại nút. Có thể hỏi nhanh: dự đoán một liên kết trích dẫn mới thuộc cấp độ nào?

Chuyển tiếp

Ngay cả ở cấp độ nút, graph cũng đặt ra một yêu cầu mà dữ liệu ảnh không gặp theo cùng cách.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216

06. Thứ tự nút không mang ý nghĩa

Thành viên 1 | 90 giây | Mốc 06:30–08:00

Trên ảnh, pixel có vị trí hàng và cột rõ ràng. Trên đồ thị, số thứ tự của nút thường chỉ là cách ta lưu dữ liệu. Nếu đổi tên nút một thành nút ba và sắp xếp lại X cùng A cho đúng, ý nghĩa của bài toán không thay đổi. Một mô hình tốt phải tôn trọng điều đó.

Công thức trên slide phát biểu tính equivariance theo phép hoán vị. P là ma trận đổi thứ tự. Khi đầu vào đổi thứ tự nút, đầu ra phân loại theo nút phải đổi thứ tự tương ứng. Ta không yêu cầu mọi hàng đầu ra giữ nguyên vị trí. Phân biệt này quan trọng, vì nếu nói đầu ra theo nút là invariant thì chưa chính xác. Khi gom toàn bộ graph để dự đoán một nhãn duy nhất, ta mới thường yêu cầu readout bất biến với thứ tự nút.

Ngoài ra, mỗi nút có thể có số hàng xóm khác nhau. Ta không thể mặc định luôn có tám vị trí xung quanh như một cửa sổ ba nhân ba trên ảnh. Vì vậy, cần một phép tổng hợp nhận được tập hàng xóm có kích thước biến đổi và không phụ thuộc thứ tự liệt kê. Tổng, trung bình hoặc cực đại là những ví dụ đơn giản. Đó là cơ sở để chuyển sang message passing.

Thao tác và nhịp trình bày

Dùng ví dụ đổi ID bài báo: bài báo vẫn là bài báo đó. Dừng 5 giây ở từ equivariance.

Chuyển tiếp

Em xin chuyển cho thành viên hai: một lớp mạng sẽ nhận và tổng hợp thông tin hàng xóm như thế nào?

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216

07. Locality và chia sẻ trọng số

Thành viên 2 | 65 giây | Mốc 08:00–09:05

Em tiếp nối từ CNN ở chương bốn. Một lớp tích chập ảnh xử lý vùng lân cận của pixel, và dùng cùng bộ trọng số tại nhiều vị trí. Hai ý tưởng đó vẫn có ích khi dữ liệu là đồ thị. Một nút tổng hợp tín hiệu từ lân cận của nó, và cùng phép biến đổi được dùng cho các nút khác.

Điểm thay đổi là cách xác định lân cận. Với CNN, ta có lưới và cửa sổ có cấu trúc cố định. Với GNN, tập hàng xóm do cạnh trong graph quyết định. Có nút chỉ có một hàng xóm, có nút có rất nhiều. Vì thế không nên hiểu rằng chỉ cần biến graph thành một ảnh rồi dùng CNN thông thường.

Bảng này là cầu nối trực giác để thấy vì sao kiến trúc phải phù hợp với cấu trúc dữ liệu. Các lớp GNN có nhiều cách xác định thông điệp và trọng số tổng hợp. GCN mà nhóm dùng là một trường hợp cụ thể. Để hiểu nó, trước hết ta nhìn cơ chế chung của message passing, rồi mới viết dưới dạng ma trận.

Thao tác và nhịp trình bày

So sánh theo từng hàng, không đọc toàn bộ bảng thành một danh sách.

Chuyển tiếp

Một lượt xử lý của nút có thể mô tả bằng thông điệp, tổng hợp và cập nhật.

Nguồn đối chiếu

Tài liệu môn Phát triển hệ thống thông minh, Trần Đình Quế, D:/School/PT HTTM/. Chương 2 đối chiếu Assignment2.pdf vì thư mục không có Slide2.pdf. Chương4/Slide4.pdf tr.3–8,28.
Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907

08. Cơ chế message passing

Thành viên 2 | 80 giây | Mốc 09:05–10:25

Giả sử ta cần cập nhật biểu diễn của nút v. Mỗi hàng xóm u đóng góp một thông điệp. Thông điệp có thể phụ thuộc vào biểu diễn của u, của v và trong một số mô hình là cả đặc trưng cạnh. Sau đó ta tổng hợp các thông điệp. Cuối cùng, một phép cập nhật tạo biểu diễn mới của v.

Các ký hiệu M, AGG và U trên slide biểu diễn những lựa chọn thiết kế, chứ không phải mọi mô hình đều dùng đúng cùng một phép tính. AGG thường phải không phụ thuộc thứ tự hàng xóm. Nếu đảo cách liệt kê u mà tổng không đổi, mô hình không bị lệ thuộc vào ID lưu trong file.

Sau một lớp, nút có thể nhận tín hiệu từ hàng xóm trực tiếp. Sau hai lớp, những hàng xóm đó đã nhận thông tin từ hàng xóm của chính chúng, nên vùng ảnh hưởng có thể tới hai bước cạnh. Độ sâu vì vậy liên quan đến phạm vi thu thập thông tin. Nhưng nhiều lớp hơn chưa chắc tốt hơn, vì thông tin có thể bị trộn quá mức hoặc việc tối ưu trở nên khó. Nhóm sẽ quay lại ý này bằng thí nghiệm độ sâu. Trước mắt, ta chọn một cách tổng hợp xác định: GCN với chuẩn hóa đối xứng.

Thao tác và nhịp trình bày

Dùng ngón tay đếm một lớp và hai lớp. Dành vài giây cho người nghe tự liên hệ tới receptive field của CNN.

Chuyển tiếp

GCN viết cơ chế đó rất gọn bằng một phép nhân ma trận.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216

09. Một lớp Graph Convolutional Network

Thành viên 2 | 90 giây | Mốc 10:25–11:55

Công thức trung tâm là H ở lớp tiếp theo bằng hàm kích hoạt của Â nhân H nhân W. H ở lớp đầu chính là X, tức các đặc trưng ban đầu. W là ma trận trọng số mà mô hình học. Â là ma trận biểu diễn cấu trúc đã thêm self-loop và chuẩn hóa.

Ta có thể đọc công thức theo hai phần. Nhân với Â làm mỗi nút nhận một tổng có trọng số từ hàng xóm và từ chính nó. Nhân với W trộn các chiều đặc trưng để tạo một không gian biểu diễn mới. Sau đó ReLU đưa tính phi tuyến vào mô hình. Vì phép nhân ma trận kết hợp được, mã nguồn có thể tính XW trước hoặc ÂX trước, miễn là thứ tự tương đối với các hàm phi tuyến vẫn đúng.

Điểm cần giữ rõ là Â tác động lên trục nút, còn W tác động lên trục đặc trưng. W dùng chung cho tất cả nút, không phải mỗi bài báo có một ma trận học riêng. Nếu thay Â bằng ma trận đơn vị, bước trộn hàng xóm biến mất. Khi đó ta có một phép biến đổi kiểu MLP áp dụng độc lập lên từng nút. Chính sự đối chiếu này giúp baseline MLP có ý nghĩa trong thực nghiệm.

Thao tác và nhịp trình bày

Chỉ Â, H và W theo đúng trục tác động. Hỏi lớp đại lượng nào được optimizer cập nhật rồi trả lời ở slide 13.

Chuyển tiếp

Vậy Â được tạo như thế nào, và vì sao không dùng trực tiếp A?

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907 Phương trình (2), (9).
Mã nguồn: demo/train.py và demo/research.py.

10. Self-loop và chuẩn hóa đối xứng

Thành viên 2 | 75 giây | Mốc 11:55–13:10

Trước tiên ta cộng ma trận đơn vị vào A. Phép cộng này thêm một liên kết từ mỗi nút tới chính nó. Nhờ vậy, khi tổng hợp hàng xóm, nút vẫn giữ một phần đặc trưng của bản thân. Nếu không có self-loop trong dạng GCN này, lớp hiện tại chỉ trực tiếp nhận tín hiệu từ các nút khác có cạnh nối tới nó.

Sau đó ta tính bậc trên ma trận đã thêm self-loop, ký hiệu là D có dấu ngã. Mỗi cạnh giữa i và j được gán hệ số một chia căn bậc hai của tích hai bậc. Đây là chuẩn hóa đối xứng. Nó điều chỉnh mức đóng góp theo cấu trúc lân cận, thay vì để tổng tăng trực tiếp theo số lượng hàng xóm.

Ta phải dùng bậc sau khi thêm self-loop, không dùng nhầm bậc của graph ban đầu. Cũng không nên gọi công thức này là trung bình cộng thông thường: tổng hệ số trên mỗi hàng không nhất thiết bằng một. Ví dụ ba nút ở slide tiếp theo sẽ cho thấy trực tiếp điều đó. Trong thư viện, GCNConv xử lý thêm self-loop và chuẩn hóa theo cấu hình. Trong bản NumPy, nhóm viết rõ phép tính để kiểm tra được.

Thao tác và nhịp trình bày

Đọc thứ tự: thêm self-loop, tính bậc, chuẩn hóa. Chỉ hệ số cạnh ở cuối slide.

Chuyển tiếp

Ta thử tính đúng một nút bằng số để công thức bớt trừu tượng.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
D:/School/PT HTTM/DeepGraph/demo/train.py.

11. Ví dụ tính tay với 3 nút

Thành viên 2 | 100 giây | Mốc 13:10–14:50

Xét ba nút A, B, C nối thành chuỗi A nối B và B nối C. Mỗi nút chỉ có một đặc trưng, lần lượt là hai, bốn và sáu. Ta đặt trọng số W bằng một để chỉ quan sát tác dụng của graph. Sau khi thêm self-loop, bậc của A và C là hai, bậc của B là ba.

Hãy tính biểu diễn mới của B. Đóng góp từ A có hệ số một trên căn sáu, vì tích bậc của A và B là hai nhân ba. Đóng góp từ chính B có hệ số một phần ba. Đóng góp từ C lại có hệ số một trên căn sáu. Vậy trước ReLU ta được hai trên căn sáu, cộng bốn phần ba, cộng sáu trên căn sáu. Kết quả xấp xỉ bốn phẩy năm chín chín. Vì giá trị dương nên ReLU giữ nguyên.

Nếu tính trung bình cộng của hai, bốn, sáu thì kết quả là bốn. Sự khác nhau này cho thấy chuẩn hóa đối xứng không phải mean thông thường. Ma trận bên trái cũng cho ta thấy A chưa nhận trực tiếp đặc trưng của C trong một lớp. Ở lớp thứ hai, A có thể nhận tín hiệu liên quan đến C qua biểu diễn đã cập nhật của B. Khi học thật, W không cố định bằng một nữa mà được tối ưu từ dữ liệu.

Thao tác và nhịp trình bày

Cho lớp 10–15 giây tự đoán bậc sau self-loop và tính hệ số của B. Chỉ lần lượt ba số hạng, dành 5 giây so sánh với mean=4.

Chuyển tiếp

Mô hình Cora dùng cùng ý tưởng này với nhiều nút và nhiều chiều đặc trưng.

Nguồn đối chiếu

Ví dụ số do nhóm xây dựng theo công thức chuẩn hóa của Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907

12. GCN hai lớp trên Cora

Thành viên 2 | 75 giây | Mốc 14:50–16:05

Ở đây X có hai nghìn bảy trăm linh tám hàng, mỗi hàng là một bài báo, và một nghìn bốn trăm ba mươi ba cột đặc trưng. Ma trận W thứ nhất biến không gian đặc trưng về ba mươi hai chiều. Sau tổng hợp và ReLU, ta có H kích thước hai nghìn bảy trăm linh tám nhân ba mươi hai.

Lớp thứ hai tiếp tục tổng hợp qua cùng graph và dùng W thứ hai đưa biểu diễn về bảy chiều. Mỗi hàng của Z lúc này chứa bảy logits. Khi cần hiển thị trên giao diện, ta áp dụng softmax để có phân bố điểm trên bảy chủ đề. Nhưng khi huấn luyện với cross_entropy của PyTorch, ta đưa logits trực tiếp vào hàm loss, không softmax lần nữa.

Hình phía dưới là minh họa kiến trúc từ bài báo gốc của Kipf và Welling. Phần điểm màu bên phải là hình của tác giả, không phải embedding từ lần chạy của nhóm. Trong code thực nghiệm, mô hình còn dùng dropout ở lớp ẩn khi train. Khi đánh giá và chạy demo, dropout được tắt. Kiểm tra các kích thước như trên là cách nhanh để phát hiện lỗi chuyển vị trọng số khi chuyển từ Python sang JavaScript.

Thao tác và nhịp trình bày

Chỉ hàng, cột của mỗi tensor và nhấn mạnh số nút không đổi qua hai lớp.

Chuyển tiếp

Bây giờ ta xem chính xác nhãn nào tham gia vào loss và mô hình học những gì.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907 Figure 1, PDF tr.4, hình gốc được trích để minh họa.
Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.

13. Huấn luyện với nhãn được phép dùng

Thành viên 2 | 115 giây | Mốc 16:05–18:00

Trong thí nghiệm này, graph A và các đặc trưng X cố định. Những gì optimizer cập nhật là các trọng số W của mạng. Ta chạy forward trên toàn graph, nhưng chỉ chọn các hàng nằm trong train mask để tính cross-entropy. Nếu vô tình dùng nhãn validation hoặc test ở bước này, kết quả sẽ không còn là đánh giá đúng quy trình.

Đoạn code thể hiện các bước chính: bật chế độ train, xóa gradient cũ, tính logits, lấy loss trên train mask, gọi backward rồi optimizer.step. Ở phía NumPy, nhóm viết đạo hàm và Adam tường minh để hiểu phép học. Ở phía PyTorch, autograd thực hiện lan truyền ngược. Hai cách triển khai hỗ trợ hai mục đích: giải thích toán và mở rộng so sánh mô hình.

Sau mỗi epoch, ta tắt dropout, tính validation loss và lưu checkpoint có loss thấp nhất. Test chỉ dùng sau bước chọn checkpoint. Việc thấy X và A của các nút test trong quá trình forward là đặc điểm của thiết lập transductive, không đồng nghĩa với dùng nhãn test để học. Nhưng nếu ta tuyên bố dự đoán cho các nút hoàn toàn chưa thấy, cần một thiết lập khác để kiểm chứng.

Nhóm đã kiểm tra gradient bằng sai phân hữu hạn. Khi sao chép cùng trọng số, sai khác logits NumPy và PyG dưới một phần tỷ. Điều đó kiểm tra cách triển khai nhất quán, chứ không thay thế đánh giá chất lượng dự đoán.

Thao tác và nhịp trình bày

Đọc 5 dòng vòng huấn luyện, chỉ rõ logits[train_mask]. Dành 10 giây hỏi tại sao model.eval() cần thiết khi validation. Có thể mở test_math.py nếu thầy hỏi sâu.

Chuyển tiếp

GCN cho ta một cách tổng hợp cụ thể. Thành viên ba sẽ giải thích những thay đổi ở GraphSAGE và GAT, rồi so sánh bằng thực nghiệm.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Mã nguồn: demo/train.py, demo/research.py, demo/test_math.py. Giá trị sai khác NumPy/PyG trong results/research/metrics.json.

14. GCN, GraphSAGE và GAT

Thành viên 3 | 65 giây | Mốc 18:00–19:05

Từ cơ chế chung vừa trình bày, ta có thể thay đổi cách tổng hợp thông tin. GCN dùng hệ số xác định từ cấu trúc và bậc nút. Cách làm tương đối gọn và phù hợp làm điểm xuất phát để hiểu phép học trên graph.

GraphSAGE nhấn mạnh việc học một hàm tạo embedding từ đặc trưng và lân cận. Nó có cơ chế lấy mẫu hàng xóm để kiểm soát lượng tính toán, và có thể được thiết kế để áp dụng cho nút chưa xuất hiện khi huấn luyện. GAT lại cho mô hình học mức đóng góp khác nhau giữa các hàng xóm thông qua attention.

Ba cách này trả lời các câu hỏi thiết kế khác nhau. Ta muốn tổng hợp theo một hệ số chuẩn hóa sẵn, muốn học hàm tổng hợp với khả năng lấy mẫu, hay muốn học trọng số attention? Chúng không tạo ra một thứ tự tốt dần chỉ vì xuất hiện ở các năm khác nhau. Trên một dữ liệu cụ thể, kết quả còn phụ thuộc đặc trưng, cấu trúc graph và cấu hình huấn luyện. Nhóm giữ cùng một quy trình thực nghiệm để quan sát sự khác nhau trong phạm vi đủ nhỏ, có thể chạy bằng CPU.

Thao tác và nhịp trình bày

Giới thiệu mỗi mô hình bằng một lựa chọn thiết kế. Không đọc tên như một danh sách lịch sử.

Chuyển tiếp

Trước hết, GraphSAGE thay cách nghĩ về việc tạo embedding cho nút.

Nguồn đối chiếu

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216
Veličković, P. et al. (ICLR 2018). Graph Attention Networks. https://arxiv.org/abs/1710.10903

15. GraphSAGE: học hàm tổng hợp

Thành viên 3 | 85 giây | Mốc 19:05–20:30

Hình này là minh họa từ bài báo GraphSAGE. Ta chọn lân cận, tổng hợp các đặc trưng từ hàng xóm, rồi dùng biểu diễn thu được để dự đoán. Thay vì gán một vector riêng có thể học cho từng ID nút, mô hình học một hàm xử lý đặc trưng và neighborhood. Do đó, nếu có một nút mới với đặc trưng và quan hệ phù hợp, ta có thể áp dụng hàm đã học để tạo biểu diễn cho nó.

Công thức trên slide mô tả biến thể mean mà nhóm dùng. Một ma trận biến đổi đặc trưng của nút trung tâm, một ma trận khác biến đổi trung bình đặc trưng hàng xóm. Hai phần được cộng rồi qua phi tuyến ở lớp ẩn. Việc tách phần bản thân và phần hàng xóm cũng giải thích vì sao số tham số của cấu hình GraphSAGE ở đây xấp xỉ gấp đôi GCN cùng độ rộng.

Cần phân biệt khả năng của kiến trúc với điều nhóm đã đo. Bài báo GraphSAGE có sampling và đánh giá inductive. Thí nghiệm của nhóm dùng toàn bộ hàng xóm và vẫn theo thiết lập transductive trên Cora. Vì vậy, bảng kết quả sắp tới chưa chứng minh được khả năng tổng quát hóa sang các nút hoàn toàn mới. Muốn kiểm chứng điểm đó, ta cần tạo split phù hợp và giữ các nút hoặc graph kiểm thử ngoài quá trình huấn luyện.

Thao tác và nhịp trình bày

Chỉ ba giai đoạn trong Figure 1. Nhấn dòng giới hạn thực nghiệm trước khi chuyển slide.

Chuyển tiếp

Nếu hàng xóm có ích ở mức độ khác nhau, GAT cung cấp một cách học các hệ số ấy.

Nguồn đối chiếu

Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216 Figure 1, PDF tr.2.
https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.conv.SAGEConv.html
Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.

16. GAT: attention trong lân cận

Thành viên 3 | 85 giây | Mốc 20:30–21:55

GAT trước hết biến đổi đặc trưng của hai nút i và j, rồi dùng một hàm attention để tạo điểm e giữa chúng. Softmax chỉ chuẩn hóa trên tập hàng xóm được xét của i, thường gồm cả i khi có self-loop. Ta thu được alpha, là hệ số dùng trong tổng hợp thông tin từ j tới i.

Khác với GCN trong thí nghiệm này, hệ số của GAT phụ thuộc vào những tham số được học và đặc trưng đầu vào. Multi-head cho phép thực hiện nhiều phép attention song song. Mỗi head có thể biểu diễn một cách kết hợp tín hiệu khác nhau. Trong cấu hình của nhóm, lớp ẩn có bốn head, mỗi head tám chiều, ghép lại thành ba mươi hai chiều. Lớp đầu ra có một head để tạo bảy logits.

Hình bên phải được trích từ bài báo gốc để minh họa nhiều head. Hình có ba head chỉ là ví dụ của tác giả, không phải cấu hình bốn head trong code của nhóm. Một lưu ý nữa là alpha lớn không tự động chứng minh quan hệ nhân quả hay cung cấp lời giải thích đầy đủ cho dự đoán. Nó cho ta một đại lượng nội bộ của phép tính. Muốn đánh giá giải thích, cần có phương pháp và kiểm thử riêng.

Thao tác và nhịp trình bày

Chỉ phạm vi softmax theo hàng xóm. Đọc phần e, alpha, rồi tổng có trọng số. Nêu rõ hình minh họa 3 head, code dùng 4 head ẩn.

Chuyển tiếp

Sau khi hiểu các kiến trúc, câu hỏi tiếp theo là chúng ta so sánh chúng trên dữ liệu nào và theo cách nào.

Nguồn đối chiếu

Veličković, P. et al. (ICLR 2018). Graph Attention Networks. https://arxiv.org/abs/1710.10903 Figure 1, PDF tr.4.
https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.conv.GATConv.html
Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.

17. Cora và quy trình đánh giá

Thành viên 3 | 110 giây | Mốc 21:55–23:45

Cora trong lần xử lý của nhóm có hai nghìn bảy trăm linh tám bài báo, một nghìn bốn trăm ba mươi ba đặc trưng nhị phân cho mỗi bài và bảy chủ đề. Nhóm chuẩn hóa từng hàng đặc trưng bằng tổng hàng. Sau loại cạnh trùng và đối xứng hóa quan hệ, graph có năm nghìn hai trăm bảy mươi tám cạnh vô hướng, tương ứng mười nghìn năm trăm năm mươi sáu cạnh hai chiều trước self-loop.

Mỗi split lấy hai mươi nhãn từ từng lớp để train, tổng cộng một trăm bốn mươi nhãn. Phần còn lại được chia thành năm trăm nhãn validation, một nghìn nhãn test và một nghìn không trăm sáu mươi tám nhãn không dùng trong loss hoặc đánh giá. Tất cả các nút vẫn tham gia cấu trúc graph. Nhóm dùng ba seed chia dữ liệu là 2026, 2027 và 2028. Với mỗi split, mỗi mạng được chạy với ba seed khởi tạo 42, 43 và 44.

Đây là thiết lập transductive: mô hình được thấy X và A của toàn bộ graph nhưng chỉ dùng nhãn train để tối ưu. Chọn checkpoint theo validation loss thấp nhất. Test không quyết định siêu tham số hay checkpoint. Cách chia này do nhóm tự xây, không phải public split của Planetoid. Vì vậy, không nên đặt số accuracy của nhóm cạnh số trong bài báo rồi kết luận mô hình nhóm tốt hơn hoặc kém hơn nếu chưa thống nhất protocol.

Thao tác và nhịp trình bày

Để lớp đọc bốn phần của split. Nhấn 1.068 nút vẫn có X và A, chỉ nhãn của chúng không tham gia đánh giá.

Chuyển tiếp

Cùng dữ liệu vẫn chưa đủ. Ta cần chỉ rõ những điều được giữ giống nhau khi so sánh.

Nguồn đối chiếu

Cora, LINQS: https://linqs-data.soe.ucsc.edu/public/lbc/cora.tgz
Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.
https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.Planetoid.html

18. Phạm vi so sánh và ngân sách học

Thành viên 3 | 85 giây | Mốc 23:45–25:10

Với các mạng nơ-ron, nhóm dùng cùng đặc trưng đã chuẩn hóa, cùng các split, Adam với learning rate không phẩy không một, weight decay không phẩy không không không năm, tối đa ba trăm epoch và early stopping với patience sáu mươi. Độ rộng lớp ẩn là ba mươi hai, dropout lớp ẩn là không phẩy năm và bias tắt. Riêng attention dropout của GAT bằng không. Logistic Regression dùng cấu hình riêng được ghi trong code vì nó có cách tối ưu khác mạng nơ-ron.

Nhóm báo cáo accuracy và macro-F1. Accuracy cho tỷ lệ dự đoán đúng trên test. Macro-F1 tính F1 cho từng lớp rồi lấy trung bình đều, nên giúp nhìn thêm chất lượng ở các lớp ít mẫu. Với nhiều lần chạy, nhóm dùng trung bình và độ lệch chuẩn để mô tả biến động.

So sánh này có giới hạn. Cùng độ rộng không đồng nghĩa cùng số tham số, như GraphSAGE có hai phép biến đổi. Nhóm cũng chưa tìm siêu tham số tối ưu riêng cho từng kiến trúc. Do đó, đây là so sánh dưới một cấu hình chung đã xác định, không phải khẳng định hiệu năng tốt nhất có thể đạt của từng mô hình. Runtime trên slide là thời gian fit trung bình trên CPU tại máy thử nghiệm, chỉ nên dùng như thông tin trong cùng môi trường.

Thao tác và nhịp trình bày

Chỉ số tham số và thời gian trong bảng. Giải thích macro-F1 bằng cách mỗi lớp có trọng số ngang nhau.

Chuyển tiếp

Trong phạm vi đó, kết quả thực tế của các mô hình như sau.

Nguồn đối chiếu

Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid. LogisticRegression(C=1000, max_iter=2000). Runtime lấy từ time.perf_counter trong fit, CPU một luồng, gồm đánh giá validation.

19. GCN đạt 80,32% accuracy

Thành viên 3 | 100 giây | Mốc 25:10–26:50

Ta bắt đầu bằng hai baseline chỉ dùng đặc trưng: Logistic Regression đạt trung bình sáu mươi mốt phẩy hai phần trăm, MLP hai lớp đạt sáu mươi phẩy bốn mươi mốt phần trăm. GCN hai lớp đạt tám mươi phẩy ba mươi hai phần trăm. Chênh lệch giữa GCN và MLP là khoảng mười chín phẩy chín mươi mốt điểm phần trăm. Cần nói điểm phần trăm, không nói tăng mười chín phẩy chín mươi mốt phần trăm theo nghĩa tương đối.

GraphSAGE đạt bảy mươi tám phẩy tám mươi bốn phần trăm, còn GAT đạt bảy mươi bảy phẩy mười hai phần trăm. Trong cấu hình này, GCN có trung bình cao nhất. Điều đó không có nghĩa GCN luôn tốt hơn các mô hình còn lại. Ta chỉ có bằng chứng cho Cora và quy trình đã nêu. GAT dùng attention vẫn có thể không cải thiện nếu dữ liệu hay cấu hình không phát huy được lợi ích của attention.

Cột độ lệch chuẩn nhắc rằng kết quả thay đổi theo split và khởi tạo. Mỗi mạng có chín runs, Logistic Regression có ba lần, một lần mỗi split. Các split có thể dùng lại cùng nút nên không phải chín bộ dữ liệu độc lập. Mean và std ở đây không phải khoảng tin cậy chín mươi lăm phần trăm. Macro-F1 cũng cao nhất ở GCN trong bảng này, củng cố quan sát nhưng chưa thay cho kiểm thử trên nhiều dataset.

Thao tác và nhịp trình bày

Chỉ khoảng cách GCN–MLP trước, rồi mới so ba GNN. Dành 10 giây để lớp nhận xét vì sao GAT không nhất thiết đứng đầu.

Chuyển tiếp

Một câu hỏi khác: giữ mô hình GCN, nếu tăng số lớp thì có tốt hơn không?

Nguồn đối chiếu

Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid. Trung bình và std dân số qua runs (numpy.std mặc định ddof=0). Chênh lệch GCN-2 trừ MLP-2 = 19,9111 điểm phần trăm.

20. Tăng độ sâu chưa cải thiện kết quả

Thành viên 3 | 70 giây | Mốc 26:50–28:00

Nhóm chạy GCN một, hai, bốn và tám lớp với cùng nguyên tắc chọn checkpoint. Accuracy trung bình lần lượt là bảy mươi hai phẩy hai mươi bốn, tám mươi phẩy ba mươi hai, bảy mươi tư phẩy bảy mươi tám và mười chín phẩy sáu mươi bốn phần trăm. Trong cấu hình này, hai lớp cho kết quả cao nhất và tám lớp suy giảm mạnh.

Quan sát là tăng độ sâu không giúp ở thiết lập đã dùng. Một giả thuyết thường được nhắc đến là oversmoothing: biểu diễn các nút bị trộn đến mức khó phân biệt. Tuy nhiên, chỉ nhìn đường accuracy đi xuống chưa đủ để xác định đây là nguyên nhân duy nhất. Các vấn đề tối ưu, dropout, lựa chọn chiều ẩn hoặc cách khởi tạo cũng có thể góp phần. Số tham số còn thay đổi theo độ sâu.

Nếu nghiên cứu tiếp, nhóm sẽ đo mức tương đồng giữa embedding, xem gradient và learning curve, thử residual connection hoặc chuẩn hóa phù hợp, rồi kiểm tra trên dữ liệu khác. Cách trình bày này tách điều đã đo khỏi điều mới là giả thuyết. Tiếp theo, thành viên bốn sẽ đưa mô hình đã học vào ứng dụng để chúng ta quan sát dự đoán từng nút.

Thao tác và nhịp trình bày

Đọc biểu đồ theo độ sâu. Nhấn mạnh 8 lớp có std lớn, không quy mọi thất bại về oversmoothing.

Chuyển tiếp

Em xin chuyển sang phần hệ thống và demo.

Nguồn đối chiếu

Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.

Li, Q., Han, Z. & Wu, X.-M. (AAAI 2018). Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. https://arxiv.org/abs/1801.07606

21. Hệ thống demo và luồng dữ liệu

Thành viên 4 | 65 giây | Mốc 28:00–29:05

Phần demo tách huấn luyện và suy luận. Ở máy local, Python đọc Cora, tiền xử lý, huấn luyện và lưu trọng số cùng dữ liệu cần thiết. Bộ thực nghiệm PyG xuất thêm metrics để so sánh. Khi mở web, trình duyệt tải các file tĩnh rồi chạy GCN bằng JavaScript trên CPU.

Mô hình chạy thật trên web là GCN NumPy hai lớp, seed bốn mươi hai, split 2026. Bảng nghiên cứu nhiều kiến trúc được nạp từ file kết quả. MLP và Logistic Regression trên màn hình đối chiếu dùng dự đoán đã lưu. Nhóm nói rõ điều này để không khiến người xem hiểu rằng trình duyệt đang huấn luyện hoặc chạy mọi kiến trúc PyG.

Lợi ích của thiết kế này là buổi trình bày không phụ thuộc vào một lượt train mới hoặc máy chủ GPU. Nhưng dữ liệu, trọng số và quy tắc chuẩn hóa phải thống nhất phiên bản. Nhóm đã đối chiếu logits JavaScript với NumPy trên toàn bộ hai nghìn bảy trăm linh tám nút, sai khác dưới một phần tỷ. Giao diện chỉ vẽ tối đa sáu mươi mốt nút lân cận để dễ xem, còn suy luận vẫn chạy trên toàn graph.

Thao tác và nhịp trình bày

Chỉ từng dòng luồng dữ liệu. Chuẩn bị tab trình duyệt đã mở sẵn trước khi chuyển slide 22.

Chuyển tiếp

Bây giờ em thử một dự đoán đúng và một dự đoán sai.

Nguồn đối chiếu

Demo và mã nguồn: D:/School/PT HTTM/DeepGraph/demo/. NumPy GCN seed 42, split 2026. URL: https://deep-graph-cora-lab.pinkwand512.chatgpt.site (private, cần tài khoản chủ sở hữu).
Các file: train.py, research.py, dist/data.json, dist/weights.json, dist/gnn.mjs, dist/research.json, test_inference.mjs.

22. Demo: dự đoán đúng và dự đoán sai

Thành viên 4 | 140 giây | Mốc 29:05–31:25

Trên giao diện, mỗi nút là một bài báo. Màu hiện tại biểu thị lớp mà GCN dự đoán. Vị trí các nút phục vụ việc vẽ lân cận, không phải embedding đã học. Em chọn tình huống GCN đúng, MLP sai, hoặc nhập mã bài báo 444191.

GCN dự đoán Neural Networks với điểm softmax khoảng ba mươi tám phẩy hai phần trăm. Em mở phần đối chiếu nhãn thật và baseline. Nhãn thật là Neural Networks, còn MLP và Logistic Regression dự đoán Theory. Đây là một trường hợp giúp quan sát khác biệt giữa mô hình dùng quan hệ và các baseline chỉ dùng đặc trưng. Một ví dụ đơn lẻ không thay thế bảng thống kê trên tập test ở phần trước.

Tiếp theo, em chọn tình huống GCN dự đoán sai, mã 158614. Nhãn thật là Theory nhưng GCN dự đoán Rule Learning. Baseline cũng sai ở ví dụ này. Trường hợp sai cho thấy mô hình vẫn có giới hạn, dù accuracy tổng thể cao hơn.

Điểm softmax không phải mức độ chắc chắn đã được hiệu chỉnh. Chẳng hạn ba mươi tám phần trăm là điểm lớn nhất trong bảy lớp và các lớp còn lại chia phần điểm còn lại. Ta không nên diễn giải nó như một cam kết rằng dự đoán này đúng với xác suất chính xác ba mươi tám phần trăm. Nhãn thật được mở để đánh giá, không được đưa vào phép tính inference của GCN.

Thao tác và nhịp trình bày

0:00–0:20 mở tab demo. 0:20–1:00 nhập 444191, mở đối chiếu, chỉ GCN và baseline. 1:00–1:40 nhập 158614, kiểm tra nhãn. 1:40–2:20 giải thích softmax, cho lớp quan sát và trở về slide. Nếu mất mạng, dùng localhost:8765 đã chạy sẵn. Ảnh trong slide là bản dự phòng của giao diện.

Chuyển tiếp

Ta vừa xem graph gốc. Nếu thay đổi các quan hệ của một nút thì điều gì xảy ra?

Nguồn đối chiếu

Demo và mã nguồn: D:/School/PT HTTM/DeepGraph/demo/. NumPy GCN seed 42, split 2026. URL: https://deep-graph-cora-lab.pinkwand512.chatgpt.site (private, cần tài khoản chủ sở hữu). Ảnh chụp ứng dụng của nhóm: results/demo-preview.png. Dự đoán từng ID đối chiếu với dist/data.json và dist/weights.json.

23. Demo: thay đổi cạnh khi suy luận

Thành viên 4 | 120 giây | Mốc 31:25–33:25

Em quay về bài báo 444191 và giữ cùng đặc trưng, cùng trọng số đã học. Trong phần thí nghiệm tương tác, em chọn bỏ năm mươi phần trăm rồi một trăm phần trăm cạnh nối trực tiếp với nút này. Ứng dụng tính lại bậc, chuẩn hóa graph và chạy GCN lại trên toàn bộ graph đã chỉnh sửa.

Mời mọi người dự đoán trước: điểm của lớp Neural Networks sẽ tăng hay giảm? Ở ví dụ này, khi bỏ toàn bộ bốn cạnh trực tiếp, điểm của Neural Networks tăng từ khoảng ba mươi tám phẩy hai lên bốn mươi lăm phẩy ba phần trăm, còn nhãn dự đoán vẫn giữ nguyên. Vì sao việc bỏ cạnh không luôn làm dự đoán tệ hơn? Hàng xóm có thể mang tín hiệu hỗn hợp, và chuẩn hóa cũng thay đổi khi cấu trúc thay đổi. Một nút riêng lẻ không bắt buộc đi theo xu hướng trung bình của cả tập test.

Ta cần gọi đúng đây là kiểm tra độ nhạy ở thời điểm suy luận. Mô hình không được huấn luyện lại. Cạnh bị bỏ theo thứ tự lưu trong dữ liệu, không phải một thí nghiệm lấy mẫu ngẫu nhiên nhiều lần. Vì vậy, kết quả này không chứng minh quan hệ nhân quả và cũng không phải bằng chứng tổng quát rằng graph là có ích hay có hại. Nó minh họa rất trực tiếp rằng dự đoán của GCN phụ thuộc vào X, A và trọng số đã học.

Thao tác và nhịp trình bày

0:00–0:25 đặt lại 444191 và chọn 0%. 0:25–0:40 hỏi lớp dự đoán chiều thay đổi. 0:40–1:15 thử 50% và 100%, quan sát điểm. 1:15–1:45 giải thích đây là inference sensitivity. 1:45–2:00 đặt lại 0%. Nếu demo không chạy, dùng số liệu đối chiếu có trên slide.

Chuyển tiếp

Để người khác có thể mở demo và kiểm tra các kết quả này, ta cần đóng gói và triển khai đúng cách.

Nguồn đối chiếu

Demo và mã nguồn: D:/School/PT HTTM/DeepGraph/demo/. NumPy GCN seed 42, split 2026. URL: https://deep-graph-cora-lab.pinkwand512.chatgpt.site (private, cần tài khoản chủ sở hữu). Hàm removeIncidentEdges trong dist/gnn.mjs. Số liệu seed42, split2026, paper444191. Giữ self-loop khi loại cạnh trực tiếp.

24. Triển khai và chạy lại thực nghiệm

Thành viên 4 | 90 giây | Mốc 33:25–34:55

Demo đã có URL triển khai. Hiện link ở chế độ riêng tư của chủ tài khoản, nên người trình bày cần đăng nhập bằng tài khoản có quyền truy cập. Trước buổi học, nhóm nên kiểm tra trên đúng máy và trình duyệt dùng để thuyết trình. Nếu muốn chia sẻ công khai, có thể đưa thư mục dist lên một dịch vụ static hosting như GitHub Pages theo hướng dẫn trong README.

Bản dự phòng local chỉ cần vào thư mục demo và chạy lệnh Python HTTP server phục vụ dist. Sau đó mở localhost cổng 8765. Toàn bộ dữ liệu và trọng số nằm trong thư mục nên không cần gọi API mô hình bên ngoài. Không nên mở thẳng index.html bằng file URL vì trình duyệt có thể chặn việc tải các file JSON.

Về tái lập, train.py là bản NumPy giúp học phép tính. research.py chạy bộ so sánh ba split và ba seed với tối đa ba trăm epoch. Các file requirements ghi phiên bản thư viện, results chứa danh sách split, metrics, learning curve và checkpoint. Để tránh gián đoạn trên lớp, ta trình bày kết quả đã chạy trước. Nếu thầy muốn kiểm tra, nhóm có thể mở một checkpoint hoặc giải thích lệnh tái chạy. Khả năng tái lập nằm ở việc lưu protocol và mã nguồn, không chỉ là đưa một ảnh chụp accuracy.

Thao tác và nhịp trình bày

Bấm liên kết demo nếu đang đăng nhập. Chỉ lệnh local và lệnh research, không cài thư viện hoặc train lại trực tiếp. Mở README trong thư mục demo khi cần hướng dẫn chi tiết.

Chuyển tiếp

Cuối cùng, em tổng hợp những giới hạn và cách nối đề tài với chương năm.

Nguồn đối chiếu

Demo và mã nguồn: D:/School/PT HTTM/DeepGraph/demo/. NumPy GCN seed 42, split 2026. URL: https://deep-graph-cora-lab.pinkwand512.chatgpt.site (private, cần tài khoản chủ sở hữu). D:/School/PT HTTM/DeepGraph/demo/DEPLOYMENT.md và README.md. https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site

25. Giới hạn và liên hệ Knowledge Graph

Thành viên 4 | 80 giây | Mốc 34:55–36:15

Cora là dữ liệu nhỏ và tương đối thuận lợi cho việc khai thác tính giống nhau giữa các nút liên kết. Trong dữ liệu khác, hàng xóm có thể thường xuyên khác nhãn, tức có tính heterophily. Khi đó, một cách tổng hợp phù hợp với Cora chưa chắc còn tốt. Graph thiếu hoặc có cạnh sai cũng làm thay đổi biểu diễn học được.

Độ sâu và quy mô là hai giới hạn khác. Thêm nhiều lớp có thể dẫn đến khó phân biệt biểu diễn hoặc khó tối ưu. Graph rất lớn làm việc tổng hợp toàn bộ hàng xóm tốn bộ nhớ và thời gian. Những hướng như sampling, chia lô hoặc kiến trúc phù hợp cần được đánh giá bằng thí nghiệm riêng, chứ chưa có trong demo nhỏ này.

Liên hệ với chương năm, Knowledge Graph cung cấp các thực thể và quan hệ có kiểu rõ ràng. GNN có thể học biểu diễn trên cấu trúc ấy để hỗ trợ phân loại thực thể hoặc dự đoán quan hệ thiếu. GraphRAG có thể khai thác graph để truy xuất ngữ cảnh cho mô hình ngôn ngữ. Tuy nhiên, demo hiện tại chỉ làm phân loại nút trên mạng trích dẫn đồng nhất. Muốn phát triển thành hệ thống tri thức, nhóm cần xử lý loại quan hệ, cách xây graph và thước đo phù hợp. Đây là hướng mở rộng có cơ sở, không phải một tính năng mà demo đã có.

Thao tác và nhịp trình bày

Liên hệ câu hỏi về chất lượng graph với phần động lực của người 1. Nếu thiếu thời gian, giữ 2 giới hạn chính và đoạn liên hệ chương 5.

Chuyển tiếp

Em quay lại câu hỏi đầu bài để chốt phần trình bày.

Nguồn đối chiếu

Li, Q., Han, Z. & Wu, X.-M. (AAAI 2018). Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. https://arxiv.org/abs/1801.07606
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216
Tài liệu môn Phát triển hệ thống thông minh, Trần Đình Quế, D:/School/PT HTTM/. Chương 2 đối chiếu Assignment2.pdf vì thư mục không có Slide2.pdf. Chương5/Slide5.pdf tr.10–14,36,51.
Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.

26. Quan hệ đã giúp ích trong thí nghiệm này

Thành viên 4 | 45 giây | Mốc 36:15–37:00

Câu hỏi mở đầu là quan hệ giữa các bài báo có giúp dự đoán chủ đề ngoài đặc trưng nội dung hay không. Trong protocol Cora của nhóm, GCN hai lớp đạt tám mươi phẩy ba mươi hai phần trăm accuracy, cao hơn MLP khoảng mười chín phẩy chín mươi mốt điểm phần trăm. Đây là bằng chứng thực nghiệm cho lợi ích của cấu trúc trong thiết lập này.

Nhóm cũng đã đi qua phép tính của mô hình, kiểm tra code, quan sát trường hợp sai và triển khai suy luận lên web. Kết luận cần giữ đúng phạm vi: kết quả phụ thuộc dữ liệu và cấu hình, tăng độ sâu hay thêm attention không tự động cải thiện. Bước tiếp theo hợp lý là thử dataset khác, kiểm chứng inductive hoặc mở rộng sang bài toán gợi ý và dự đoán quan hệ. Nhóm xin cảm ơn thầy và các bạn, và xin dành phần còn lại cho câu hỏi.

Thao tác và nhịp trình bày

Dành 45 giây chốt ý rồi chuyển sang 3 phút hỏi đáp. Cả bốn người cùng trả lời theo phần phụ trách.

Chuyển tiếp

Câu hỏi thảo luận: Nếu graph có nhiều cạnh sai, nhóm sẽ thiết kế một thí nghiệm nào để kiểm tra ảnh hưởng?

Nguồn đối chiếu

Kết quả chạy ngày 13/10/2026: D:/School/PT HTTM/DeepGraph/demo/results/research/metrics.json. Cấu hình và cách chia tập: research.py. Đây là split tùy chỉnh, không phải public split Planetoid.

Kipf, T. N. & Welling, M. (ICLR 2017). Semi-Supervised Classification with Graph Convolutional Networks. https://arxiv.org/abs/1609.02907
Hamilton, W. L., Ying, R. & Leskovec, J. (NIPS 2017). Inductive Representation Learning on Large Graphs. https://arxiv.org/abs/1706.02216
Veličković, P. et al. (ICLR 2018). Graph Attention Networks. https://arxiv.org/abs/1710.10903

Câu hỏi phản biện (1/2)

1. Vì sao thấy node test trong graph mà vẫn được coi là đánh giá hợp lệ?

Vì protocol là transductive: được dùng X và A của toàn graph, chỉ dùng nhãn train trong loss. Validation chọn checkpoint. Test chỉ đánh giá sau đó. Nếu mục tiêu là các nút chưa xuất hiện hoặc dự đoán tương lai, phải thiết kế inductive hoặc temporal split tương ứng.

Người trả lời gợi ý: Thành viên 3

2. Tại sao không so trực tiếp với accuracy trong bài báo GCN?

Nhóm dùng split tùy chỉnh, cách đối xứng hóa và siêu tham số được ghi trong code. Khác split có thể khác độ khó. Muốn so trực tiếp phải thống nhất dữ liệu, tiền xử lý, split, ngân sách tìm siêu tham số và cách tổng hợp kết quả.

Người trả lời gợi ý: Thành viên 3

3. GAT mới hơn, tại sao kết quả thấp hơn GCN?

Đã đo dưới một cấu hình chung, chưa tune riêng từng kiến trúc. Attention làm tăng khả năng biểu diễn nhưng không đảm bảo cải thiện trên mọi dữ liệu. Kết quả này chỉ xác nhận GCN có trung bình cao nhất trong các lần chạy hiện tại.

Người trả lời gợi ý: Thành viên 3

4. GCN 8 lớp thất bại có chứng minh oversmoothing không?

Chưa. Accuracy thấp có thể liên quan cả tối ưu, dropout, biểu diễn và cấu hình. Cần đo tương đồng embedding, phân tích gradient hoặc learning curve, rồi thử các thay đổi có kiểm soát để phân biệt nguyên nhân.

Người trả lời gợi ý: Thành viên 2 hoặc 3

5. GraphSAGE có phải mô hình duy nhất xử lý được nút mới?

Không. GraphSAGE nhấn mạnh học hàm tổng hợp có thể áp dụng cho nút mới và cơ chế sampling. Một phép GCN cũng có thể áp dụng với đặc trưng và cấu trúc mới nếu hệ thống và protocol phù hợp. Demo hiện tại chưa đánh giá inductive cho bất kỳ mô hình nào.

Người trả lời gợi ý: Thành viên 2 hoặc 3

6. Vì sao bỏ cạnh mà điểm softmax lại tăng?

Mỗi hàng xóm có thể đóng góp tín hiệu khác nhau. Bỏ cạnh còn làm thay đổi chuẩn hóa theo bậc. Ở node 444191, điểm Neural Networks tăng nhưng đó là một trường hợp với trọng số cố định. Không thể suy ra rằng bỏ cạnh nói chung làm mô hình tốt hơn.

Người trả lời gợi ý: Thành viên 4

Câu hỏi phản biện (2/2)

7. Trọng số nào được học? Có học ma trận A không?

Trong mô hình đang chạy, X và A cố định, optimizer học các ma trận biến đổi W và tham số attention nếu là GAT. GAT học hệ số attention trên những cạnh được xét, không đồng nghĩa học tự do toàn bộ cấu trúc A.

Người trả lời gợi ý: Thành viên 2

8. Số 5.278 cạnh khác một số tài liệu Cora là do đâu?

Đây là số cạnh vô hướng duy nhất sau xử lý của code hiện tại. Quan hệ hai chiều được lưu thành 10.556 cạnh có hướng trước self-loop. Số được trích ở nguồn khác có thể dùng quy tắc đếm hoặc tiền xử lý khác. Cần xem chính file và quy trình thay vì so các số rời rạc.

Người trả lời gợi ý: Thành viên 1 hoặc 3

9. Có chứng minh GNN luôn tốt hơn MLP không?

Không. Chênh lệch 19,91 điểm phần trăm là kết quả trên Cora trong protocol đã nêu. Muốn kết luận rộng hơn cần thêm dữ liệu, nhiều điều kiện graph và ngân sách tune hợp lý. Graph có cạnh sai hoặc heterophily có thể khiến tổng hợp lân cận không phù hợp.

Người trả lời gợi ý: Thành viên 3

10. Website đang chạy mô hình gì, có gọi API không?

JavaScript chạy GCN hai lớp với trọng số từ NumPy seed42, split2026. Nó tải file dữ liệu và trọng số tĩnh, không gọi API LLM. MLP/LR dùng dự đoán đã lưu để đối chiếu. Bảng PyG là báo cáo từ thực nghiệm offline.

Người trả lời gợi ý: Thành viên 4

11. Mean ± std có phải khoảng tin cậy 95% không?

Không. Std ở đây là numpy.std với ddof=0 qua các runs. Các split có thể trùng node. Báo cáo này mô tả biến động theo split và khởi tạo, chưa cung cấp phép suy luận thống kê về mọi dataset.

Người trả lời gợi ý: Thành viên 3

12. Nếu có thêm thời gian, nhóm sẽ nghiên cứu gì?

Ưu tiên một thay đổi có giả thuyết rõ: thêm dataset có heterophily, hoặc đánh giá inductive, hoặc kiểm tra tỷ lệ cạnh nhiễu với nhiều seed. Giữ nguyên nguyên tắc chọn mô hình bằng validation và chỉ báo cáo test sau khi đã chốt cấu hình.

Người trả lời gợi ý: Cả nhóm